Fewer Bytes, Not Fewer FLOPs: Training Language Models Cheaply on CPUs

Nikunj Mehndiratta Draft, 28 October 2026. Preprint in preparation (arXiv style).

Abstract

Training a language model on a CPU is never cheap per FLOP. It is cheap only when the total FLOP budget is small, the hardware is already owned, and the budget goes into work that moves few bytes and reaches quality in few tokens. Our running example is a 15 W-class Intel Core Ultra 7 255U laptop. Training a 19M-parameter transformer stack in PyTorch, it sustains about **88 GFLOP/s of effective throughput with PyTorch's default layer and 150 to 230 GFLOP/s with dropout turned off**. That is between one six-hundredth and one fifteen-hundredth of a single A100 running llm.c. It means a GPT-2-124M run on 10B tokens would take one to three years instead of 90 minutes and about $20 (llm.c #481). We survey where CPU training time goes and find that the dominant losses are not arithmetic. Our own profile (experiment E2) finds a surprise: **dropout's random-number generation alone takes 35% to 41% of a default PyTorch training step**, because it runs on one thread. With dropout off, GEMMs take 65% of the step, attention 11%, framework overhead 9%, and the optimizer about 2%. A thread and clock sweep (E1) shows that an earlier "2048² GEMMs run 3.7× slower" reading was a transient, not a cache or bandwidth limit. PyTorch's default threading beats every hand-pinned core set, and throughput drifts down by up to a quarter over a quarter of an hour on battery. The literature adds hybrid-core load imbalance, power limits and memory-bound non-GEMM operators. The best GPU profiling study we know of puts about 39% of BERT training time in operators that account for 0.2% of the FLOPs (Ivanov et al.). We then build a taxonomy of methods that claim to make training cheaper and check, for each, whether it saves *training* compute, *training* memory traffic, or only *inference* cost, and what CPU evidence exists. Most celebrated "CPU-efficient" methods save inference only: ternary BitNet weights, fast feedforward trees, LookupFFN, MADDNESS and bitnet.cpp all fall in this group. The methods with real training leverage on a laptop are of three kinds. First are those that cut the number of tokens needed: narrow synthetic data, compute-optimal sizing, up to four epochs of reuse, and modern optimizer and architecture recipes. Second are those that cut bytes moved per step: operator fusion, 8-bit or factored optimizer states, and fused per-layer updates. Third is coarse, hard-routed conditional computation. Low-precision arithmetic helps only on CPUs with AVX512_BF16 or AMX, which consumer Intel laptops lack. We close with a CPU-first training recipe and twelve experiments, each sized to fit on the laptop, and report the first two.

1. Introduction: a laptop that trains at about a thousandth of a GPU

The question this paper answers is simple: what makes training language models cheap on CPUs? The honest first answer is that almost nothing makes it cheap compared with a GPU. The second answer is more useful. Within a fixed and small compute budget, some techniques multiply what a CPU can achieve several-fold, and many popular techniques do nothing for training at all.

The question matters to us because our own work is CPU-shaped. **Grove**, our library, uses ternary BitLinear layers stored at two bits per weight, a tree-routed feed-forward network (soft routing during training, one leaf per token at inference), a product-key memory layer, and numpy-only CPU inference. **cpulm** is a 10.7M-parameter model trained from scratch on Colab for maths word problems. It scores 13.4% to 18.0% on SVAMP; the 18.0% run adds a learned reranker over beam candidates, according to our project records. Both projects were designed around cheap *inference*. This paper asks whether the same design choices make *training* cheap, and finds mostly that they do not.

We make five contributions. Section 3 decomposes where CPU training time goes, using our laptop measurements as a worked example. Section 4 gives a taxonomy of training-cost-reduction methods,

separating training savings from inference savings and recording the CPU evidence for each. Section 5 records the status of CPU training frameworks as of October 2026. Section 6 gathers every published CPU-vs-GPU training number we could find, with its provenance. Section 7 proposes a CPU-first recipe and a set of laptop-runnable experiments, and reports the first two: a thread, clock and power sweep (E1) and a per-operator profile of a training step (E2). We found no published per-operator profile of transformer training on a CPU before E2.

Evidence conventions. Cited facts carry an inline link. Statements marked **[M]** are our own measurements on the laptop described below. Statements marked **[I]** are our own derivations, inferences or proposals; they are not taken from any source. A dagger (†) marks a cited item that our research notes record as not re-fetched from the primary source, taken from a search summary or secondary source, or otherwise unverified. Claims marked "vendor" were published by a company selling the hardware or software. Unless stated otherwise, published speedups were measured on GPUs.

---

2. Background: FLOP accounting, rooflines, and the test machine

2.1 The 6ND rule and what it leaves out

The standard estimate of training compute is **C ≈ 6·N·D** FLOPs, for N parameters and D tokens (Hoffmann et al. 2022†). The 6N per token splits into three parts, each about 2N: the forward matmuls, the backward matmuls for activation gradients, and the backward matmuls for weight gradients. **[I]** That split matters throughout Section 4. A method that only cheapens the forward pass touches about a third of training FLOPs. A method that freezes weights (as LoRA does) removes about a third. A method that quantizes weights but must still form a dense weight gradient leaves at least a third untouched. The rule ignores attention-score FLOPs, normalisations, elementwise operations and the optimizer step. Those ignored items are exactly where CPUs lose time.

2.2 The roofline

Attainable throughput is the minimum of peak compute and memory bandwidth multiplied by arithmetic intensity (FLOPs per byte of DRAM traffic). Kernels to the left of the ridge point are memory-bound (Williams, Waterman & Patterson 2009). **[I]** For a square fp32 GEMM of size n, intensity is n/6 FLOP/byte: about 171 at n = 1024 and about 341 at n = 2048. For our model's layer shapes (2,048 tokens per step, d = 512, FFN width 2,048), the attention projections come out at about 114 FLOP/byte and the FFN matmuls at about 171. A layer norm, softmax, activation, bias or residual sits at roughly 1 FLOP/byte or below. AdamW's update sits near 0.5: it moves about 28 bytes per parameter (reading weight, gradient, m and v; writing weight, m and v) for 10 to 15 FLOPs.

2.3 The test machine

The Intel Core Ultra 7 255U is an "Arrow Lake" U-series part. It has 2 P-cores, 8 E-cores and 2 low-power E-cores (12 cores and 14 threads, so only the P-cores are hyperthreaded). Turbo clocks are 5.2, 4.2 and 2.4 GHz respectively. It has a 12 MB L3, **15 W base power and 57 W maximum turbo power**, and dual-channel LPDDR5/X-8400 or DDR5-6400 memory (Intel specification page). Arrow Lake-U is refreshed Meteor Lake silicon on Intel 3 (Wikipedia: Arrow Lake)). Its LP E-cores have a private 2 MB L2 and **no access to the L3**. They see about 175 ns DRAM latency, against about 153 ns from the main E-cores (Chips and Cheese; Chips and Cheese, "No L3 Hurts").

Intel's specification page lists only SSE4.1, SSE4.2 and AVX2. Notebookcheck additionally lists AVX-VNNI and FMA3 (Notebookcheck). Either way, the chip has **no AVX-512 and no AMX**. Its 12-TOPS NPU and 8-TOPS iGPU ratings are INT8 inference figures (Intel). The machine has 16 GB of RAM and no discrete GPU.

2.4 Our measurements

[M] Our first measurement, on 28 October 2026, was a single short run. It found fp32 matmul in PyTorch 2.14 (CPU build, MKL and oneDNN, 12 threads by default) at **434 GFLOP/s for 1024×1024** and **118 GFLOP/s for 2048×2048**. With the same stack we trained a 6-layer transformer (d = 512, FFN 2,048, batch 8 × 256 tokens, fp32 eager mode, AdamW) at **about 775 tokens/s**. That model used PyTorch's default `TransformerEncoderLayer`: post-norm, ReLU, **dropout 0.1**, no attention mask.

[I] The parameter count, $6 \times (4 \cdot 512^2 + 2 \cdot 512 \cdot 2048) \approx 18.9$M, matches the transformer blocks alone. The measurement therefore contains no embedding or vocabulary projection, and a real LM head would add cost on top. By the 6N rule, 6 × 19M × 775 ≈ **88 GFLOP/s effective**. Attention-score matmuls add roughly 8% more real work at T = 256. A step of 2,048 tokens costs about 2.3×10^{11} parameter FLOPs.

[M] Experiments E1 and E2 (Section 7.3) repeated these measurements with logging. Two results change how the first figures should be read. The **2048^2 figure did not reproduce**: interleaved and sustained 2048^2 GEMMs ran at 265 to 379 GFLOP/s, close to 1024^2. The **775 tokens/s figure did reproduce** (764 and 961 tokens/s in two E2 runs), but a third or more of that step was dropout's random-number generation. Without dropout, with pre-norm, GELU and a causal mask, the same stack trained at 1,352 to 2,044 tokens/s, or **150 to 230 GFLOP/s effective**. The spread comes from the machine's state: heat and the battery's power limits.

The throughput sets the laptop's compute budget. **[I]** At a planning rate of 180 GFLOP/s, one day is about 1.6×10^{16} FLOPs. **1e15 FLOPs is about 1.5 hours, 1e16 about 15 hours, and 1e17 about 6.4 days.** With PyTorch's default dropout layer, each figure is about twice as long.

---

3. Where CPU training time actually goes

3.1 Memory-bound operators take a third or more of the step while doing almost none of the arithmetic

The best operator-level profile of transformer training is Ivanov et al.'s study of a BERT encoder layer. It was measured on V100 GPUs, not CPUs. Tensor contractions were **99.80% of FLOPs but only 61.0% of runtime**. Normalisations (softmax, LayerNorm) were 0.17% of FLOPs and 25.5% of runtime. Elementwise operations were 0.03% of FLOPs and 13.5% of runtime (Ivanov et al., MLSys 2021). The paper's introduction says "37%" of time is memory-bound while its table sums to 39%, and it does not reconcile the two. The non-GEMM operators ran at 0% to 1.3% of peak; even the GEMMs reached only 55% to 62%, and the batched attention GEMMs only 16% to 22%. Fusion and layout changes cut data movement by up to 22.91% and sped up the encoder layer 1.30× (1.19× for full BERT) (Ivanov et al.).

We found **no equivalent per-operator breakdown for transformer training on any CPU**, so we measured one (E2, Section 7.3). PyTorch's own tuning guide recommends CPU training mainly for "small-scale or memory-bound models such as DLRM" (PyTorch Performance Tuning Guide).

[M] The CPU picture differs from the GPU one in an unexpected way:

- **With PyTorch's default layer, dropout dominates.** One operator, `bernoulli_`, which draws dropout masks, took **35% to 41% of the step**: 0.94 to 0.99 s of 2.1 to 2.9 s. All GEMMs together took 39% to 45%.
- The cause is a serial random-number generator. **[M]** Drawing 62.9M masks, about one step's worth, took 424 ms on 1 thread and 421 ms on 12.
- Ivanov et al.'s GPU profile has no dropout share this large, because GPU random-number generation is parallel.
- **Without dropout, the step looks like the GPU profile.** GEMMs took 65%, the attention kernel 11%, framework overhead outside operators 9%, copies and layout changes 4%, other elementwise operators 4%, and LayerNorm only 1%.

- **[I]** The memory-bound share is smaller than on the V100 (about 35% of time, against 39%). The CPU's low ridge point (3 to 6 FLOP/byte, from 434 GFLOP/s over an estimated 70 to 130 GB/s) makes GEMMs relatively slower, so they take a larger share.
- The parameter GEMMs ran at about 240 GFLOP/s inside the step. That is 55% to 65% of the square-GEMM rate, the same fraction Ivanov et al. report for GPU GEMMs.

[M] The optimizer is negligible at this scale: AdamW took 32 to 37 ms, or 1.3% to 2.4% of the step. That matches our estimate **[I]** of about 0.53 GB moved per step for 19M fp32 parameters. The optimizer becomes significant only with very small batches or very large sparse tables (Section 4.3).

[I] The levers for this stack, in order:

1. Turning off dropout (1.65× to 1.8× measured).
2. Fusing the remaining 20% of non-GEMM time plus the 9% overhead, worth at most about 1.4×.
3. Faster GEMMs, which are bounded by clocks and power (Section 3.2).

3.2 The apparent 3.7× drop from 1024^2 to 2048^2 was a transient; clocks, power and scheduling set the GEMM rate

[I] At 171 and 341 FLOP/byte, both GEMM sizes sit far above the laptop's ridge point, so DRAM bandwidth could not explain a 3.7× slowdown at 2048^2. **[M]** E1 shows there is no stable slowdown to explain:

- **2048^2 is close to 1024^2.** Interleaved with other sizes, 2048^2 ran at 356 to 379 GFLOP/s against 437 to 438 for 1024^2. Sustained for 30 s, 2048^2 ran at 265 to 373 against 282 to 368.
- Rerunning the original snippet twice gave 234 and then 428 GFLOP/s for 2048^2.
- The 118 figure was a single 3-second reading taken while the chip's state was changing.
- **Size matters little from 1024^2 to 4096^2.** Every size in that range ran within about 25% of the others under PyTorch's default threading.

Before E1, our notes ranked four candidate explanations: power, hybrid straggling, cache capacity and SMT contention. **[M]** E1's data bears on them as follows.

The first is the **power budget**, and it shows up as drift rather than as a size effect. The same default configuration measured at the start and end of the 16-minute battery sweep lost:

- 23% of sustained 1024^2 GEMM rate (368 → 282 GFLOP/s);
- 27% of training throughput (2,044 → 1,497 tokens/s).

Mean P-core clocks during training fell from 3.39 to 2.82 GHz over the same period. The chip's 15 W base and 57 W turbo limits, and an OEM sustained limit we have not read, are the likely cause **[I]**.

The second is **hybrid straggling**. Intel's oneMKL guide warns that when P- and E-cores share work, "static load balancing … is likely to result in lower performance because E-cores will take longer to complete the work items assigned to them". It says dynamic balancing pays off only on very large problems, and static balancing on P-cores alone often wins for small or regular problems. All-core execution may still win when E-cores greatly outnumber P-cores, as they do here (Intel oneMKL Developer Guide). On this chip the slowest threads run on LP E-cores that cannot use the L3. **[M]** E1 agrees on the LP-E cores. Fourteen threads, which spill onto the LP-E cores, trained at 1,120 tokens/s, the worst of all settings. PyTorch's default of twelve trained at 1,497 to 2,044. But E1 contradicts the advice to drop the E-cores. Pinned to the two P-cores alone, the stack trained at 1,228 tokens/s; pinned to the eight E-cores alone, 1,388.

The third is **cache capacity**. The three 2048^2 matrices total 48 MB, four times the 12 MB L3, while the 1024^2 set is about 12 MB. **[M]** 3072^2 and 4096^2 ran as fast as 1024^2 under default threading (411 to 458 GFLOP/s), so cache capacity does not set a cliff in this range.

The fourth is **SMT contention**. Two of the 14 threads are hyperthread siblings sharing a P-core's FMA units. **[M]** Adding the SMT siblings to the two P-cores left training unchanged: 1,205 against 1,228 tokens/s.

PyTorch's guide calls `OMP_NUM_THREADS` "the easiest switch" for CPU speed. It recommends thread pinning and `KMP_BLOCKTIME` of 0 or 1, and notes that Intel's OpenMP and jemalloc or tcmalloc can help. It gives no advice specific to hybrid cores (PyTorch Tuning Guide). A community note warns that thread oversubscription can cause "dramatic" slowdowns (Coderx7 gist†).

[M] On Windows, **hand-pinning hurt**. We set the process affinity mask before PyTorch loaded and matched the thread count to the cores. Every pinned set was slower than PyTorch's unpinned default of twelve threads:

- 10 P+E cores, our recipe's earlier advice: 1,226 tokens/s.
- 12 P+SMT+E cores: 1,604 tokens/s.

The logged clocks suggest why. The two pinned sets that mixed P- and E-cores averaged 2.6 GHz on their P-cores (2.4 to 2.9) and drew 16 to 28 W of system power. The unpinned twelve-thread default averaged 3.25 GHz (2.8 to 3.5) and drew 28 to 41 W. Unpinned fourteen threads also ran slowly (2.7 GHz). **[I]** We do not know why these settings run at lower clocks. Windows' hybrid scheduling policy is the obvious suspect, and a mains-powered repeat (pending) would separate it from battery limits.

PyTorch 2.14's CPU build reports MKL 2026.1 and oneDNN 3.12 with OpenMP threading. We have not read the OEM's configured power limits.

3.3 The theoretical peak is far above anything sustainable at 15 W

[I] Our notes estimate a full-turbo, all-core fp32 peak of about **850 GFLOP/s**. The estimate assumes two 256-bit FMA units per P-core (32 FLOP/cycle) and two 128-bit FMA pipes per E-core (16 FLOP/cycle). These widths come from general microarchitecture knowledge and were not verified against a source. On this estimate:

- our best GEMM rate (434 to 458 GFLOP/s) is about half of burst peak;
- sustained GEMMs (265 to 373 GFLOP/s) reach 31% to 44% of burst peak;
- the dropout-free training rate (150 to 230 GFLOP/s) reaches 18% to 27%;
- the default-layer rate (88 GFLOP/s) reaches about 10%.

[M] Measured against the best GEMM rate, dropout-free training reaches 35% to 50% model-FLOPs utilisation. That is about what E2's breakdown (65% of the step in GEMMs running at about 60% of the square-GEMM rate) implies.

3.4 The 255U has no hardware path for low-precision training

Consumer Intel chips from Meteor Lake to Arrow Lake and Lunar Lake disable AVX-512 because their E-cores lack it (Wikipedia: Arrow Lake)). Their AI-related CPU instructions are 256-bit AVX-VNNI integer dot products. Pre-launch Intel documents add AVX-VNNI-INT8, AVX-IFMA and AVX-NE-CONVERT on Arrow Lake-S/H and Lunar Lake (TechPowerUp†). None of these gives a native bf16 multiply-accumulate.

Native bf16 compute on CPUs needs AVX512_BF16, found on Cooper Lake, Sapphire Rapids and later Xeons, and on AMD Zen 4 and Zen 5. Matrix-unit speed needs AMX. AMX shipped in Sapphire Rapids Xeons in January 2023, supports only BF16 and INT8, and does "2048 INT8 or 1024 BF16 operations per cycle" per core (Wikipedia: AMX; Intel AMX article). No consumer chip has AMX.

AMD's Zen 4 runs AVX-512 "double-pumped" over 256-bit units. Desktop and server Zen 5 has a full 512-bit datapath. Laptop Zen 5 ("Strix Point") keeps 256-bit datapaths but still exposes AVX-512 including BF16 (TechPowerUp). Apple's M4 reaches **over 2.3 TFLOPS fp32** through SME outer products. Its BF16 and FP16 SME throughput is "comparatively low", and only INT8 gives about a 2× gain (Remke & Breuer, "Hello SME!"). Hugging Face warns that AVX2-only CPUs "are not guaranteed to have better performance" under IPEX (HF:

Efficient Training on CPU).

[I] On the 255U, bf16 autocast would add conversion work to fp32 arithmetic. It might shave a little traffic from memory-bound operators but should not speed up matmuls.

Chip family	AVX-512	Native bf16 compute	int8 dot product	Matrix unit
Meteor Lake / Arrow Lake-U (our 255U)	No	No	AVX-VNNI (256-bit; vendor page omits it)	None
Lunar Lake / Arrow Lake-S/H	No	No (AVX-NE-CONVERT converts only)	AVX-VNNI, AVX-VNNI-INT8†	None
AMD Zen 4	Yes (double-pumped)	AVX512_BF16	AVX-512 VNNI	None
AMD Zen 5 desktop/EPYC	Yes (512-bit)	AVX512_BF16	AVX-512 VNNI	None
AMD Zen 5 laptop	Yes (256-bit datapath)	AVX512_BF16	AVX-512 VNNI	None
Xeon Sapphire Rapids and later	Yes	AVX512_BF16 + AMX-BF16	VNNI + AMX-INT8	AMX
Apple M4	n/a	SME bf16, slow	SME I8, about 2×	SME, about 2.3 TFLOPS fp32

Table 1. CPU instruction support relevant to training, compiled from the sources in Section 3.4. Lunar Lake and Arrow Lake-S/H entries rest partly on pre-launch documents.

3.5 Bandwidth is the ceiling for everything that is not a GEMM

[I] Both memory options on the 255U use a 128-bit bus. That gives a theoretical **134 GB/s at LPDDR5X-8400 and 102 GB/s at DDR5-6400**. Sustained bandwidth on client chips is typically well below theoretical; our notes give 60% to 80% as an unsourced rule of thumb. We have not measured STREAM bandwidth on this laptop or checked its installed memory speed. Meteor Lake's memory controller idles in a low-power state unless the CPU tile demands bandwidth (Chips and Cheese). On the related Arrow Lake-S, one Lion Cove core reaches only about 54 GB/s of L3 read bandwidth (Chips and Cheese). Memory-bound operators therefore need many threads, which runs straight into the hybrid-scheduling problem above.

For comparison, Meta reports a custom memory-layer kernel reaching **about 3 TB/s on an H100**, against under 400 GB/s for stock PyTorch (Berges et al.). **[I]** That puts the laptop's bandwidth gap to a datacenter GPU at more than an order of magnitude. Its compute gap (Section 6) is larger still, at about three orders.

[I] The overall picture is this. On a laptop CPU, GEMM arithmetic is not the binding constraint for a small model. The binding constraints are the power envelope, the hybrid-core scheduler, the activation traffic of unfused elementwise and normalisation operators, and the fixed overhead of many small kernel calls. Techniques that attack those constraints pay. Techniques that only make multiply-accumulates cheaper pay little on this hardware.

---

4. A taxonomy of methods, sorted by what they save in training

Table 2 is the paper's central summary. Each family is scored on whether it reduces training compute, training memory or traffic, and inference cost, and on what CPU *training* evidence exists. The subsections give the evidence.

Family	Cuts training FLOPs?	Cuts training memory / traffic?	Cuts inference?	CPU training evidence	Fit for the 255U [I]
bf16 mixed precision	Per-FLOP cost, on AVX512_BF16/AMX only	Yes (about half of activations)	Yes	Vendor: 1.4–1.6× on Cooper Lake; DistilBERT fine-tune 8× vs Ice Lake	Poor (no native bf16)

Family	Cuts training FLOPs?	Cuts training memory / traffic?	Cuts inference?	CPU training evidence	Fit for the 255U [I]
INT8 training (SwitchBack, Jetfire)	13–42% faster on GPU	Yes	Yes	None found	Speculative (AVX-VNNI, no framework)
FP8 / FP4 training	Yes on GPU	Yes	Yes	None (no CPU format)	None
Ternary / 1-bit (BitNet b1.58, TriLM, MatMul-free)	No measured saving; dense weight gradient remains	Up to 61% memory (fused GPU kernels)	Yes, large	One self-reported claim (0.5–0.7× speed)†	Inference only
MoE / hash layers	FLOPs to reach a quality (Switch up to 7×)	No (more parameters)	Per token, yes	None found	Coarse experts with hard routing only
Fast feedforward / tree FFN (soft training)	No (all leaves evaluated)	No	Yes (78× FFN layer on CPU)	Trained on GPU	Only if training is hardened
Product-key / memory layers	Yes, per parameter	Sparse rows, if the optimizer is sparse	Yes	None found	Promising
SLIDE / BOLT (LSH neurons)	Yes, in huge sparse output layers	Partly	Yes	Yes (vendor and author): 2–15× per epoch vs TF on V100	Narrow (output softmax only)
LookupFFN, MADDNESS	No (soft or fixed operands)	No	Yes	Inference only	Inference only
meProp, RigL	On paper	Little	Some	Small models only; minimal gains for LLMs	Weak
MeZO / zeroth-order	No: about 18× more wall-clock time (derived)	Yes, up to 12×	No	None found	Small fine-tunes, non-differentiable rewards
LoRA / QLoRA / PEFT	About 6N → 4N per token	Yes, large	No	Vendor Xeon fine-tunes; no laptop tokens/s	Useful for borrowed models of 100M or more
Data efficiency (TinyStories, Cramming, speedrun, distillation)	Yes: fewer tokens to target	n/a	Sometimes (smaller model)	Hardware-agnostic; none run on CPU	Highest leverage
Optimizer and memory tricks (8-bit Adam, Adafactor, Lion, LOMO, GaLore)	Rarely (speed claims shrink under fair tuning)	Yes, 50–90% of optimizer state	No	ZeRO-Offload CPU Adam (optimizer step only)	Useful when tables are large

Table 2. Taxonomy of training-cost reductions. Cells summarise Sections 4.1–4.8; the final column is our judgement.

4.1 Low-precision and integer training pays only where the silicon supports it

Mixed precision (fp16 or bf16 arithmetic with an fp32 master copy and loss scaling) halves weight and activation memory and depends on tensor-core-class hardware (Micikevicius et al.†). On CPUs, the evidence comes almost entirely from Xeons. PyTorch's blog reports bf16 at **1.4× to 1.6×** over fp32 for DLRM, ResNet-50 and ResNeXt-101 on Cooper Lake with AVX512_BF16 (PyTorch blog). A Hugging Face and Intel post reports DistilBERT fine-tuned on SQuAD for one epoch in **about 26 minutes on one Sapphire Rapids instance** using AMX, bf16 and IPEX, against 3 hours 30 minutes on Ice Lake. Four nodes took 7.5 minutes (HF blog). **[I]** That "8×" compares a new matrix unit against an older CPU probably running fp32. It is not a bf16-versus-fp32 comparison on the same chip, and it includes no GPU baseline. Intel's own AMX training article describes FP32, AVX-512 BF16 and AMX BF16 comparisons but publishes no numbers (Intel). An Intel/TUM report says AMX helps mainly at batch sizes of 64 and above and little at 1 to 8 (Intel/TUM report†, vendor-linked, no supporting benchmark seen).

Integer training works on GPUs at modest scale. SwitchBack's int8 linear layer gives **13% to 25% speedups** while matching bf16 within 0.1 points on a 1B-parameter CLIP ViT-Huge, which its authors call "the largest int8 training to date". Their analysis also links loss spikes to AdamW underestimating the second moment (Wortsman et al.). Jetfire keeps the whole dataflow in INT8 with per-block quantisation. It reports a **1.42× end-to-end speedup and 1.49× memory reduction** per transformer block against FP16 (Xi et al., ICML 2024†). Its key point is that it attacks memory-access overhead, the same bottleneck CPUs face. INT4 training (Xi et al. 2023†), FP8-LM (39% less memory and 75% faster than BF16 Megatron on H100) (Peng et al.), FP8 at 2T tokens on Gaudi2 (Fishman et al.†) and NVFP4 pretraining of a 12B model on 10T tokens (NVIDIA†) all rely on GPU or accelerator number formats that CPUs lack.

We found **no LM-scale INT8, FP8 or INT4 training on any CPU**, and no framework that performs true int8 training on CPUs. **[I]** AVX-VNNI could in principle run a SwitchBack-style int8 forward and input-gradient GEMM on the 255U. However, Jetfire's lesson cuts both ways: a quantise-GEMM-dequantise sequence adds traffic, which a bandwidth-bound CPU can least afford.

4.2 Ternary and 1-bit weights save inference, not training

BitNet b1.58 2B4T was trained on 4T tokens. Its released bf16 checkpoint is "the master weight … used for training only", and weights are ternarised by absmean in the forward pass. The report gives **no training compute or cost comparison** with full precision (Ma et al. 2025). Its savings are inference savings: 0.4 GB of non-embedding memory against 1.4 to 4.8 GB, and **29 ms CPU decode latency per token against 48 ms and 65 ms** for LLaMA 3.2 1B and Qwen2.5 1.5B on an i7-13800H (Ma et al.). The report's two benchmark tables give inconsistent averages (54.19 and 55.01).

TriLM ternary models outperform float and post-quantised models of the same bit size above 1B parameters, but report no training cost (Kaushal et al., Spectra). The MatMul-free LM is the one result with a measured *training* saving: its fused GPU implementation "reduces memory consumption by up to 61% during training" (Zhu et al.). On CPUs, bitnet.cpp gives **2.37× to 6.17× inference speedups** on x86 (Wang et al.). The only CPU ternary *training* claim we found is a self-reported model page. It states that the weight gradient remains an "FP32 outer product" and reports training at about **0.5× to 0.7×** of the dense speed (AlphaNeural Chimera 5.1†).

[I] The mechanism is clear from the three matmuls of a BitLinear step. The forward pass $Y = X \cdot Q(W)$ and the input gradient $dX = dY \cdot Q(W)^T$ can use ternary weights; the latter needs dY quantised. The weight gradient $dW = X^T \cdot dY$ is a dense real-valued product under the straight-through estimator. That bounds the arithmetic saving to about two thirds of the matmul FLOPs, before any quantisation overhead. Meanwhile the optimizer still reads and writes full-precision latent weights and states.

For Grove, this means the 2-bit storage format is an inference economy. The one training-side saving worth pursuing is to quantise each latent weight once per step and reuse the two-bit copy for both the forward and input-gradient products. That cuts weight-read traffic, not FLOPs.

4.3 Conditional computation saves training compute only when routing is hard and coarse

Mixture of experts keeps FLOPs per token fixed while adding capacity. Switch Transformer reports up to a **7× pre-training speedup** over T5 "while still using the same FLOPS per token", on TPUs (Fedus et al.). DeepSeekMoE 16B matches LLaMA2 7B with about **40% of the compute** (Dai et al.). The saving is in compute to reach a quality, not in per-token cost below a dense model of equal active size. Finer-grained experts become "increasingly memory bandwidth bound" even on H100s because of gather and scatter (SonicMoE). ScatterMoE trims that overhead to 66.2% of MegaBlocks' training memory (Tan et al.). Hash Layers route each token by a fixed hash of its id, with no router parameters and no balancing loss. They are competitive with Switch and BASE layers (Roller et al., NeurIPS 2021). We found **no MoE training on CPUs**. **[I]** Since fine-grained MoE is already bandwidth-bound on HBM, it will be worse on a laptop. Coarse experts executed as dense GEMMs over token groups, and statically hashed routing that lets tokens be pre-bucketed, are the CPU-friendly end of the design space.

Fast feedforward networks (FFF) are log-time *only at inference*. In training, each node outputs a soft split, the output is a mixture of all leaves, and the cost scales with the number of leaves (Belcak & Wattenhofer). FFF's abstract claims up to 220× over a dense feed-forward layer. The paper's tables show at most 3.93×, measured on an A100, and all timings are inference (FFF paper). UltraFastBERT was trained for **a day on a single A6000 GPU**. Its CPU result is that the FFN layer alone runs 78× faster than a dense GEMM baseline on an i7-6700HQ. On GPU its PyTorch implementation was 3.15× faster than the fastest fused dense kernel. It keeps at least 96% of BERT-base's GLUE score, but CoLA falls from 51.3 to 35.1 for the long-trained model (Belcak & Wattenhofer). The authors note that no native conditional matmul exists and that attention is untouched.

[I] Grove's tree FFN inherits this training-cost profile. With soft routing over all leaves it costs at least as much as a dense FFN of the same total width. Making it cheap to train requires hard top-1 or top-few-path routing during training, with tokens bucketed by leaf so each leaf runs as a dense GEMM. None of the cited papers evaluates that.

Product-key and memory layers are the most CPU-plausible sparse layer we found. Product keys make exact top-k search over N keys cost about 2√N comparisons, and only a few value rows receive gradients per token (Lample et al. 2019). Meta's memory layers scale this to 128B memory parameters on 1T tokens. An 8B model with memory reaches 63.04 on MMLU against 59.68 dense, and 1.3B Memory+ approaches Llama2-7B on factual QA (Berges et al.). Meta describes the layer as "almost entirely memory bandwidth bound" and needed custom kernels (about 6× faster end-to-end EmbeddingBag) to train it. The paper reports no total GPU-hours against dense (Berges et al.). **[I]** Each gather and scatter touches a contiguous row of 512 to 2,048 values, which suits CPU prefetchers. The hidden cost on a CPU is the optimizer. Dense Adam over a million-row value table touches every row each step unless a lazy sparse optimizer updates only the rows touched. For Grove's PKM, that is probably the largest single training-cost lever. No CPU training measurement of PKM exists.

4.4 Hash and lookup training works on CPUs, but in one narrow regime

SLIDE is the best-known demonstration that CPUs can beat GPUs at *training*. Each layer's neurons are indexed in LSH tables, only the retrieved neurons are computed forward and backward, and threads update asynchronously. On a 44-core Xeon it reached **over 3.5× faster time-to-accuracy than TensorFlow on a V100** (about 1 hour against 3.5 hours) (Chen et al., MLSys 2020). The setting is extreme multi-label classification. Over 99% of compute is in a 205K- or 670K-way output layer, fewer than 0.5% of output neurons are active, and inputs average about 75 non-zeros.

Optimized SLIDE, an Intel collaboration, reports **2.0× to 15.5× per-epoch speedups** over TensorFlow on a V100 using Cascade Lake and Cooper Lake Xeons (for example 3.5× and 7.8× on Amazon-670K, 9.2× and 15.5× on Text8). AVX-512 alone contributed only 1.12× to 1.22×, and most of the gain came from memory and layout work. Its caveats are significant. The GPU baseline was out-of-the-box dense TensorFlow, comparisons are per

epoch rather than time-to-accuracy, Amazon-670K's batch had to be raised to 1,024, and the paper's BF16 table contradicts its prose (Daghaghi et al.). MONGOOSE learns the hash functions and schedules table rebuilds, reporting 6.5× speedup and 6× memory reduction against prior LSH methods. Its comparisons are on the GPU side, not CPU-versus-GPU (Chen et al., ICLR 2021). We found **no third-party reproduction or published rebuttal** of the SLIDE family's CPU-beats-GPU claims.

ThirdAI's BOLT engine productised these ideas. An AWS co-authored post reports BOLT on an Ice Lake CPU taking 1,470 s against 5,130 to 7,550 s for dense PyTorch and TensorFlow on a T4G GPU, all at 33.6% accuracy on Amazon-670K. BOLT itself was not run on a GPU (AWS ML blog, vendor/partner). ThirdAI's September 2023 press release claims **BOLT2.5B was pre-trained in about 20 days on 10 Sapphire Rapids CPUs**, with "preliminary" quality on par with GPT-2 XL (press release reprint, vendor). We found no paper, independent evaluation or open-weights benchmark of it, so it remains unverified.

The other lookup methods are inference techniques. LookupFFN replaces FFN GEMMs with learnable hash lookups, trained through a softmax relaxation, and targets CPU inference (Zeng et al., ICML 2023). MADDNESS reports about 100× faster approximate matmul on CPUs but assumes a fixed operand (Blalock & Guttag). Sparse-backprop and sparse-training methods save FLOPs on paper. meProp updated 1% to 4% of weights per step without accuracy loss on small models (Sun et al.), but a later paper found it overfits at 50% sparsity in some settings (SWAT). The Gopher authors found RigL-style sparse training gave "minimal reduction in the FLOPs required" for LLMs (Rae et al.).

[I] The transferable lessons from SLIDE are engineering, not hashing: compute only the selected rows forward and backward, update without locks, use hugepages and cache-friendly layout, and rebuild indices rarely. Grove's tree router and PKM select rows deterministically, so they avoid the LSH-maintenance cost that MONGOOSE identifies. The LM output softmax over a large vocabulary is the one transformer component where SLIDE-style sampling has a demonstrated CPU training precedent.

4.5 Forward-only training trades memory for an order of magnitude more time

MeZO estimates gradients from two forward passes with a shared random perturbation. It reports **up to 12× memory reduction**, full fine-tuning of a 30B model on one A100 where backpropagation fits 2.7B, and results comparable to backprop fine-tuning on several tasks up to 66B (Malladi et al., NeurIPS 2023). ZO-Bench quantifies the cost. On full OPT-13B fine-tuning, ZO-SGD needed 64 GB on one A100 at 11.1 s per iteration, against 148 GB and 19.1 s for first-order SGD. But it ran **20,000 iterations against 625**. With LoRA, a 5 to 6 point gap to first-order methods persisted on harder tasks. Without careful prompt formatting, ZO-SGD lost about 10 points on SST-2 where first-order lost 0.1 (Zhang et al., ICML 2024).

[I] Those numbers imply about **18× more wall-clock time** for zeroth-order training to reach a lower accuracy, though batch sizes and stopping rules may differ between the arms. Forward-Forward has no LM-scale results. A 2026 study found its memory advantage did not hold at realistic scale (secondary host†). Zeroth-order methods are therefore a **fine-tuning** tool for memory-starved settings and for non-differentiable objectives, not a pretraining method. **[I]** MeZO also does not combine directly with two-bit weights, because tiny perturbations need a full-precision latent copy. Its plausible use in our work is fine-tuning against a checker-based reward on Grove programs.

4.6 Parameter-efficient fine-tuning cuts memory far more than compute

LoRA freezes pretrained weights and trains low-rank adapters, reducing GPT-3's trainable parameters about 10,000-fold and GPU memory about 3-fold (Hu et al.†). QLoRA backpropagates through a frozen 4-bit base into LoRA adapters and fits a 65B fine-tune on one 48 GB GPU (Dettmers et al. 2023†). **[I]** Per token, LoRA still needs the forward pass and the activation-gradient pass and skips only the frozen weight gradients. That is about 4N instead of 6N FLOPs, a 1.5× compute saving. The large saving is the gradients and AdamW state for frozen weights. QLoRA adds dequantisation work and is usually slower per token; its benefit is memory, which is rarely binding for 0.1B to 1B models on a 16 GB laptop.

CPU evidence is thin and mostly from vendors. Lenovo and Intel describe LoRA on 5th-generation Xeon with AMX as "a reasonable alternative" to GPUs but give no tokens/s figure (Lenovo Press LP2179). An individual blog reports Llama-3-8B PEFT on a Xeon 6443 running "6-7x" faster with AMX than without, "approaching" A100 time, with absolute times only in a chart (ss8319 blog). One 2025 paper avoids gradient training on laptops altogether by composing adapters from a bank of pre-trained LoRAs. The results beat the base model but fall short of GPU-trained adapters (Arabpour et al.). For scale, LoRA on Qwen2.5-1.5B on an RTX 4060 GPU runs at about 360 to 628 tokens/s (arXiv 2509.12229).

[I] At the dropout-free planning rate of 180 GFLOP/s and about 4N FLOPs per token, the laptop would run LoRA at roughly **450 tokens/s on a 0.1B model, 90 on 0.5B and 45 on 1B**. Each rate is about half that with dropout on. So 1e16 FLOPs buys about 25M fine-tuning tokens at 0.1B, or 5M at 0.5B. These are estimates, not measurements.

4.7 Data and recipe efficiency are the largest multipliers, and they are hardware-agnostic

Narrow data beats scale at tiny budgets. TinyStories trained models of 1M to 35M parameters on GPT-3.5 and GPT-4-generated stories written in about a 1,500-word child vocabulary, with a 10K-token vocabulary. Each model took "at most 30 hours" on one V100. Models under 10M parameters, or with a single transformer block, produced fluent and consistent stories. A 28M model compared favourably with GPT2-XL on that domain, while 125M models trained on web text "can rarely generate" coherent stories (Eldan & Li). Grammar emerged first. Consistency appeared when hidden size grew from 64 to 128, and depth mattered more than width for consistency and instruction-following.

TinyGSM applied the same lever to maths: 12.3M GPT-3.5-generated grade-school problems with Python solutions. A 1.3B generator plus a 1.3B verifier reached **81.5% on GSM8K**, above its teacher's 77.4% (Liu et al.). Phi-1 reached 50.6% on HumanEval at 1.3B using textbook-quality and synthetic data (Gunasekar et al.†). **[I]** cpulm is the TinyStories-to-TinyGSM transfer at a hundredth of the scale. The precedent argues that simplifying and templating the problem distribution, with programmatic solutions, is worth more than any arithmetic trick.

Make each step cheaper, do not shrink the model. Cramming trained BERT for one day on one consumer GPU, using 5 to 13 exaFLOPs, and reached **78.3 to 78.6 average GLUE against 80.9** for full BERT-base. The original recipe given the same day reached about 52 (Geiping & Goldstein). Its gains came from pre-LayerNorm, removed biases, a gated FFN, a one-cycle schedule peaking at 1e-3, a growing batch size, no dropout, and filtering of hard-to-compress data. Its key negative finding is that "per-token efficiency of training depends strongly on model size, but not transformer type": smaller models learn less per token, which "largely mitigates any throughput gains" (Geiping & Goldstein).

Modern recipes cut tokens-to-target by over 30×. The modded-nanoGPT speedrun cut the tokens needed for GPT-2-small-level FineWeb loss (3.28) from 10B to under 330M, and wall-clock time on 8×H100 from 45 minutes to 0.665 minutes by 30 August 2026 (modded-nanogpt). The early, hardware-agnostic steps were LR tuning, rotary embeddings, the Muon optimizer, ReLU², QK-norm, zero-initialised projections, an untied head, value embeddings, U-net skips and smaller batches. Later steps such as FP8, FlexAttention and CUDA graphs are GPU-specific. Moonshot reports Muon needs about half of AdamW's FLOPs at LLM scale (Liu et al. 2025†). A careful benchmark finds that, under fair tuning, scalar optimizers such as Lion give under 1.2× over AdamW, and matrix optimizers such as Muon and SOAP give about 1.3× below 520M parameters, falling to 1.1× at 1.2B. Tuning AdamW's learning rate alone can give 2× over common defaults (Wen et al., ICLR 2026†).

Size for the budget and reuse data. Chinchilla's roughly 20 tokens per parameter (Hoffmann et al.†) gives the following when extrapolated **[I]**: about 2.9M parameters on 58M tokens at 1e15 FLOPs, 9.1M on 183M at 1e16, and 28.9M on 577M at 1e17. These extrapolations come from fits made several orders of magnitude higher. At this size embeddings dominate the parameter count, which is a known source of disagreement between scaling fits (Pearce & Song†; Porian et al.†). Data repetition is nearly free up to a point: "up to 4 epochs of repeated data yields negligible changes to loss compared to having unique data" (Muennighoff et

al.).

Distillation works; curriculum mostly does not. Across BabyLM 2023's more than 30 submissions, distillation gave strong results, and curriculum learning, tried by many teams, mostly failed to help (BabyLM 2023 findings). The 2024 winner was a hybrid causal and masked objective, GPT-BERT, and performance tracked training FLOPs strongly (Hu et al. 2024; Charpentier & Samuel). The 2025 round found that relationship weaker and that interaction with a teacher "can yield high-quality models" (BabyLM 2025). Baby Llama, a 58M student distilled from teachers trained on the same 10M words, beat its teachers (Timiryasov & Tastet†). Reported compute savings for on-policy distillation range from 9× to 100× but come only from secondary summaries (Maginative†).

[I] On a CPU, the cheapest form of distillation is offline. Generate a synthetic corpus once, or cache top-k teacher logits, and then train the student at unchanged cost per token. Running a 125M teacher live on a 10M student would add about four times the student's training FLOPs per token.

4.8 Optimizer and memory tricks cut traffic, not arithmetic

Fp32 AdamW keeps 8 bytes of state per parameter and moves about 28 bytes per parameter per step **[I]**. Several methods cut that. Eight-bit optimizers quantise m and v block-wise and "maintain 32-bit performance with a small fraction of the memory footprint" on models up to a 1.5B LM (Dettmers et al. 2021). Adafactor factors the second moment into row and column statistics (Shazeer & Stern†). Lion keeps only momentum (Chen et al.†). LOMO fuses the gradient computation with the update so full gradients are never stored. It uses 10.8% of the memory of a standard DeepSpeed setup and fine-tunes a 65B model on eight RTX 3090s (Lv et al.).

GaLore projects gradients into a low-rank subspace. It cuts optimizer memory by up to 65.5% (82.5% in 8-bit) and pretrains a 7B model on a 24 GB GPU (Zhao et al.). But its periodic SVD makes it far slower in amortised terms; one comparison measures 13.72× slower than LoRA including projection updates (SparseLoRA appendix). Gradient checkpointing trades about 30% extra compute for O(√n) activation memory (Chen et al. 2016†).

On CPUs, the only established result is ZeRO-Offload. It runs the Adam step on the host CPU with a vectorised CPU Adam while forward and backward stay on a GPU. It trains models over 10B parameters on one V100 at 40 TFLOPS (Ren et al.). **[I]** For a 19M dense model the optimizer is under 1% of step time, so these tricks matter little. They matter a great deal for million-row PKM tables and for 100M-plus models fine-tuned on a laptop. LOMO-style fusion, which consumes each layer's gradient while it is still in cache, is the trick most closely matched to a bandwidth-bound CPU. Checkpointing is rarely needed on CPUs, which usually have enough RAM for activations.

---

5. Frameworks and kernels: upstream PyTorch is the only well-supported path

PyTorch with oneDNN and Inductor. By late 2026, mainstream CPU training runs on upstream PyTorch. The Inductor CPU backend in PyTorch 2.1 uses oneDNN for GEMMs and convolutions and generated C++ for elementwise and reduction operators. Its announcement measured inference, not training (PyTorch dev-discuss). Intel's AMX guide uses `torch.cpu.amp.autocast()` "whether you are using Intel Extension for PyTorch or not". oneDNN picks AMX by default, and `ONEDNN_MAX_CPU_ISA` can override the choice (Intel). We found **no 2024 to 2026 post measuring CPU *training* speedups from** `torch.compile`, which is why Section 7 includes that experiment.

Intel Extension for PyTorch is end-of-life. Intel "discontinued active development" and stopped releases after version 2.8, announced on 10 October 2025. It said most features had been upstreamed into PyTorch, and the repository was archived on 30 March 2026 (IPEX issue #867). Guides that rely on `ipex.optimize` describe a retired path.

DeepSpeed uses the CPU mainly for optimizer offload. Its CPU Adam is "5X–7X faster than the standard PyTorch implementation", by DeepSpeed's own account (DeepSpeed tutorial, vendor). It is not a full CPU training system.

llama.cpp and ggml removed their `finetune` and `train-text-from-scratch` examples in July 2024 (PR #8669 mirror). They then added a `ggml-opt` training API (commit mirror). The current README calls fine-tuning "technically functional (for FP32 models and limited hardware setups)" but "very much WIP" (Debian copy of the README). An open issue from December 2025 reports failures even on a 17M-parameter model (issue #18499 mirror). Note that both sources for the removal and re-addition are mirrors, not the upstream repository.

llm.c offers "a simple reference CPU fp32 implementation in ~1,000 lines" (llm.c README). Apple's MLX fine-tuning numbers we found run on the Apple GPU, not the CPU, and come from low-quality sources (InsiderLLM†). We did not research JAX, ONNX Runtime Training or MLX's CPU device.

Stack	CPU training status, Oct 2026	bf16	int8 training
PyTorch eager + oneDNN	Mainstream, supported	Autocast; fast only with AVX512_BF16/AMX	No
PyTorch `torch.compile` (Inductor CPU)	Supported; training speedups unpublished	Yes	No
Intel IPEX	EOL after 2.8; archived 2026-03-30	Was the main AMX path	No (QLoRA-style only in IPEX-LLM)
DeepSpeed	CPU Adam for offload only	n/a	No
llama.cpp / ggml	Experimental, FP32-only, reported failures	No	No
llm.c (CPU path)	Reference fp32 C/OpenMP	No	No

Table 3. CPU training framework status, compiled from the sources in Section 5.

[I] For training a small transformer from scratch on a laptop in 2026, the practical choice is plain PyTorch, optionally compiled, or a hand-written C/numpy path. The specialised CPU-training tools are retired (IPEX), experimental (ggml) or proprietary (BOLT).

---

6. Honest numbers: CPU training runs about three orders of magnitude behind a GPU

6.1 Throughput

The cleanest small-scale CPU reference is llm.c's fp32 C path. On a MacBook Pro M3 Max with eight OpenMP threads, it fine-tunes GPT-2 124M at about **1.3 s per 256-token step** (llm.c README). **[I]** That is about 195 tokens/s, or about 147 GFLOP/s by the 6N rule. nanoGPT's CPU "baby GPT" (4 layers, width 128, 2,000 iterations) runs in "about ~3 minutes" (nanoGPT README). **[I]** That works out to roughly 8,500 tokens/s and about 40 GFLOP/s, a rough figure that includes evaluation time. On the GPU side, a single A100 40GB ran llm.c's GPT-2 124M at **177,968 tokens/s at 49% MFU**, and an 8×A100 node trained it on 10B tokens in about 90 minutes for about $20 (llm.c #481). **[I]** That is about 133 TFLOP/s per A100.

System	Workload	Stack	Reported	Effective rate [I]	Source type
Core Ultra 7 255U laptop	19M transformer stack, PyTorch default layer (dropout 0.1)	fp32 PyTorch eager	764–961 tok/s **[M]**	about 88–109 GFLOP/s	Our measurement (E2)
Core Ultra 7 255U laptop, battery	Same stack, no dropout, pre-norm, causal	fp32 PyTorch eager	1,352–2,044 tok/s **[M]**	about 150–230 GFLOP/s	Our measurement (E1, E2)

System	Workload	Stack	Reported	Effective rate [I]	Source type
M3 Max CPU, 8 threads	GPT-2 124M fine-tune, B=4, T=64	fp32 llm.c	about 1.3 s/step	about 195 tok/s, 147 GFLOP/s	llm.c
Unspecified CPU	0.8M char-GPT	fp32 PyTorch	about 3 min, loss 1.88	about 8.5k tok/s, 40 GFLOP/s	nanoGPT
1× A100 40GB	GPT-2 124M	llm.c bf16	177,968 tok/s	about 133 TFLOP/s	llm.c #481
8× A100 80GB	GPT-2 124M, 10B tokens	llm.c	about 90 min, about $20	about $2 per billion tokens	llm.c #481
Sapphire Rapids, 1 node	DistilBERT SQuAD, 1 epoch	AMX bf16 + IPEX	about 26 min (8× vs Ice Lake)	—	HF/Intel, vendor co-authored
GCP C3, 176 vCPU	RoBERTa-125M SQuAD	AMX	under 25 min, 4.1–4.5× vs N2	under about $3.1 per run	The Register, cloud vendor
Xeon cluster (MLPerf v2.1/v3.0)	BERT-large	AMX bf16	47.26 / 47.93 min; 31.06 min open on 16 nodes	—	TechPowerUp, DigiTimes, vendor

Table 4. Published and measured training throughput. Node counts for the closed-division MLPerf runs were not verified on mlcommons.org.

[I] With dropout off, our 150 to 230 GFLOP/s matches or beats llm.c's CPU path on a high-end M3 Max (147 GFLOP/s), though llm.c's figure is for a 124M model at a much smaller batch. With PyTorch's default dropout, our 88 GFLOP/s sits between the nanoGPT and llm.c figures. Against a single A100, the laptop is about **1/600 to 1/900 as fast without dropout, and 1/1,500 with it**. Matching llm.c's GPT-2 run ($6 \times 124\text{M} \times 10\text{B} \approx 7.5 \times 10^{18}$ FLOPs) would take about **1 to 1.6 years** of continuous laptop time without dropout, and 2.7 years with it.

6.2 Cost

No rigorous, independent, like-for-like cost-per-token comparison of CPU and GPU transformer training exists in the sources we found. Intel's pro-CPU cost argument concerns classical machine learning: an A100 instance must be at least 1.6× faster to break even against a 64-vCPU Xeon instance (Intel Analytics Software, vendor). It does not address deep-learning training.

[I] A rough electricity calculation is still informative. 1e16 FLOPs on the laptop takes about 15 hours without dropout. **[M]** During E1, the whole system drew 28 to 33 W on battery while training at the default setting. **[I]** That comes to about 0.4 to 0.5 kWh for the run. One A100 at llm.c's efficiency would do the same work in about 75 seconds. At the roughly $14/hour 8×A100 rate, that costs a few cents. Even the laptop's marginal electricity is therefore comparable to or above the rental cost of a GPU doing the same FLOPs. The caveat is that a 10M to 20M model will not reach 49% MFU on an A100; a community benchmark found an A100 twice as cost-efficient as an H100 on nanoGPT for exactly this reason (HF forum). CPU training wins on cost only when the hardware is owned and otherwise idle, the model is too small to keep a GPU busy, or no GPU is available.

6.3 What has actually been trained on CPUs

The documented record is short. In 2012, a 1B-connection sparse autoencoder was trained on 16,000 cores across 1,000 machines for three days and learned cat-face detectors (Le et al.). Xeon clusters trained BERT-large and ResNet-50 to MLPerf targets in 2022 and 2023, and Intel noted it "remains the only CPU vendor to submit MLPerf results" (Intel newsroom, vendor). The SLIDE and BOLT family trained extreme classifiers, and ThirdAI claims BOLT2.5B (unverified, Section 4.4). Single-node CPU successes are fine-tunes: DistilBERT, RoBERTa-125M, a Stable Diffusion textual inversion on 4 nodes of dual Xeon 8480+ (200 steps in about 5 minutes) (HF/Intel), and Llama-3-8B PEFT. ZeRO-Offload uses the CPU only for the optimizer step. **We found no credible 2024 to 2026 from-scratch pretraining of a competitive LLM on CPUs alone.**

---

7. A CPU-first recipe and experiments that fit on one laptop

7.1 The recipe

The recipe below is our proposal **[I]**. It assembles published pieces that have not been tested together, ordered by expected leverage.

First, fix the machine and the framework defaults before touching the model.

- **Turn dropout off.** On this CPU it costs 1.65× to 1.8× in throughput (E2), and small models trained for one or a few epochs rarely need it; Cramming also drops it.
- If regularisation is needed, prefer weight decay or more data.
- If dropout is unavoidable, apply it to few tensors, never to the attention probabilities.
- **Leave PyTorch's default threading alone.** On Windows, its default of twelve threads beat every hand-pinned core set by 20% to 45% (E1).
- Do not use all fourteen threads; the extra two land on the slow low-power E-cores.
- **Train plugged in**, on a performance power plan, with adequate cooling. Expect throughput to drift down by up to a quarter over the first quarter-hour on battery (E1).
- **Keep GEMM dimensions at 512 to 4,096 and multiples of 64.** Every size in that range ran within about 25% of the best.
- **Try** `torch.compile` to fuse the roughly 20% of the dropout-free step spent outside GEMMs and attention, plus the 9% framework overhead (E3).

Second, spend the FLOPs on the right model and data. Size the model near the Chinchilla extrapolation for the budget: about 9M parameters for a 1e16 run (about 15 hours without dropout). Count the embeddings, and prefer a small vocabulary. Use at least four layers. Keep sequences at 128 to 256 tokens. Narrow the distribution with templated, programmatically solved problems in the style of TinyGSM. Reuse data for up to four epochs, then synthesise more rather than repeat further.

Third, adopt the hardware-agnostic parts of the speedrun recipe. These are pre-norm with QK-norm, no biases, a ReLU² or gated FFN, zero-initialised output projections, value embeddings, a tuned one-cycle or warmup-stable-decay schedule with a high peak, and small batches. Benchmark Muon against a properly tuned AdamW instead of assuming the published speedups.

Fourth, cut bytes where the bytes are large. Use fused or vectorised optimizer updates. Use 8-bit or factored optimizer states and a lazy sparse update for any large lookup table such as Grove's PKM, and fuse per-layer updates LOMO-style once models reach 100M or more.

Fifth, use sparsity only where it is coarse and hard. Train Grove's tree FFN with hard top-1 or top-2 path routing and leaf-bucketed GEMMs instead of soft all-leaf mixing. Consider fixed hash routing as a no-router baseline.

Sixth, move compute to where it is cheapest. Pretrain on a free or rented GPU when one exists, as cpulm already does on Colab. Use the laptop for data generation, evaluation, verifier-reranking, offline distillation targets and small fine-tunes.

Finally, avoid these on this chip: bf16 autocast for speed, ternary weights for training speed, MeZO for pretraining, SLIDE-style hashing inside transformer blocks, and curriculum ordering as a primary lever.

7.2 Experiments

Each experiment below is sized to the laptop and resolves an open question in this paper. Times are our estimates at 88 GFLOP/s; without dropout they are about half.

ID	Question	Procedure	Laptop cost	Decision it informs
E1	Is the 2048² slowdown power, hybrid scheduling or cache?	Interleave GEMM sizes 256–4,096; sweep core sets {2 P, 4 P+SMT, 8 E, 10 P+E, 12 P+SMT+E pinned; 12 and 14 unpinned}; log clocks and power; on mains and on battery	**Battery half done** (16 min, Section 7.3); mains half pending	Thread count and power settings; whether sustained GEMM rate is near 434 or 118
E2	Where does our training step go on a CPU?	`torch.profiler` on the 19M stack: GEMMs, attention, norms, elementwise, optimizer, framework overhead	**Done** (Section 7.3)	Fills the missing CPU counterpart of Ivanov et al.'s breakdown
E3	Does Inductor fusion speed up CPU training?	Eager vs `torch.compile` on the same model, after confirming the Windows build supports CPU compilation (unverified)	About 1 hour	Whether fusion is the main software lever
E4	Which shapes maximise utilisation?	Sweep d {256, 384, 512, 768} and tokens per step {1k, 2k, 4k, 8k}; report tokens/s and FLOP/s	About 2 hours	Default width and batch for all later runs
E5	Does optimizer traffic matter at our scale?	AdamW (default, foreach, fused) vs Adafactor vs Lion vs 8-bit Adam (if bitsandbytes runs on this CPU, unverified); equal tokens	About 3 hours	Optimizer default; expected near-zero effect for dense 19M
E6	Can Grove's tree FFN be cheap to train?	Soft all-leaf vs hard top-1 and top-2 path routing with leaf-bucketed GEMMs vs dense FFN of equal active width; compare loss at equal FLOPs and equal wall-clock	About 1e15 FLOPs per arm (about 3 hours)	Whether to harden routing during training
E7	Does a sparse optimizer make PKM cheap to train?	Dense Adam vs lazy row-sparse update on a PKM with 64K–1M values; measure step time and loss	About 3 hours per arm	PKM optimizer design
E8	What does ternary training cost on this CPU?	BitLinear (STE, absmean ternary, int8 activations) vs fp32 linear: step time and loss at equal tokens	About 2 hours	Whether ternary QAT should be a late fine-tuning phase only
E9	Does the Chinchilla extrapolation hold at 1e15?	Train 3M, 5M, 9M and 15M models on cpulm's maths data for 1e15 FLOPs each; repeat the best with 1, 2, 4 and 8 epochs	About 12 hours, plus about 12 hours	Model size and data reuse for 1e16 runs

ID	Question	Procedure	Laptop cost	Decision it informs
E10	Which recipe pieces transfer to CPU?	At 1e15: baseline AdamW vs tuned LR vs modern architecture vs Muon vs a GPT-BERT mixed objective	About 3 hours per arm	Default recipe
E11	Is fine-tuning a pretrained model better than training from scratch?	At equal 1e16 FLOPs: from-scratch cpulm vs LoRA on a pretrained 0.1–0.5B model; evaluate on SVAMP with the same reranker	About 1.3 days per arm	Whether cpulm should stay from-scratch
E12	Can zeroth-order fine-tuning use a checker reward?	MeZO fine-tune of cpulm on programs scored by Grove's verifier vs supervised fine-tuning on verified beams	About 1 day	Whether ZO earns a place in self-training

Table 5. Proposed laptop experiments. All costs are estimates.

E1 and E2 come first because they decide whether the remaining experiments should be read as software results or power results. E9 and E11 decide the most money-relevant question: whether this project's compute should go into from-scratch models at all.

7.3 First results: E1 and E2

All runs were on 28 October 2026, on battery, on the "Samsung Mode" power plan, with PyTorch 2.14.1 (CPU build, MKL 2026.1, oneDNN 3.12, OpenMP). The code is in `cputrain/` alongside this paper:

- `topology.py`, `model.py`, `e1.py`, `e2.py` and `summarise.py`;
- raw results in `cputrain/results/`.

Core topology. Windows reports logical CPUs 0–1 and 10–11 as the two hyperthreaded P-cores, 2–9 as the eight E-cores (two L2 clusters of four), and 12–13 as the low-power E-cores. Windows puts the E and LP-E cores in the same efficiency class. The LP-E cores are the two outside the shared 12 MB L3.

Procedure (E1). For each core set, a fresh process was pinned before PyTorch loaded, with the thread count matched to the set. Each process ran three phases:

1. GEMMs at seven sizes, interleaved in shuffled order over three rounds (median reported).
2. Thirty seconds each of 1024^2 and 2048^2 GEMMs, in 5-second windows.
3. Twenty seconds of training the dropout-free 19M stack.

Every second, a logger recorded each logical CPU's effective clock (Windows' processor-performance counters) and the system's power draw (the battery's discharge rate). There was a 15-second rest between sets, and the first set was repeated at the end to measure drift. A first attempt was discarded because the laptop entered Modern Standby mid-run.

Core set	256^2	512^2	1024^2	2048^2	4096^2	Sustained 1024^2	Sustained 2048^2	Train tok/s	P-core GHz (train)	System W (train)
12 threads, default (first)	372	422	437	356	458	368	373	**2,044**	3.39	33.2
2 P	156	190	197	184	200	181	196	1,228	4.14	17.8
4 P+SMT	181	201	216	266	260	200	217	1,205	4.05	31.2
8 E	276	304	332	305	364	267	255	1,388	2.92	29.0
10 P+E	215	267	371	242	444	243	223	1,226	2.36	16.4
12 P+SMT+E	214	239	243	239	362	273	289	1,604	2.87	27.6
14 threads, unpinned	165	195	317	239	323	196	203	1,120	2.65	20.5

Core set	256^2	512^2	1024^2	2048^2	4096^2	Sustained 1024^2	Sustained 2048^2	Train tok/s	P-core GHz (train)	System W (train)
12 threads, default (repeat)	329	410	438	379	411	282	265	**1,497**	2.82	27.6

Table 6. E1 on battery. GEMM columns are fp32 GFLOP/s. Training is the dropout-free 19M stack. System power is the battery's discharge rate, which the firmware smooths over tens of seconds, so per-phase watts are approximate. One 5-second window of the 2 P set stalled (a 2 GFLOP/s reading with a matching gap in the log); the median of six windows absorbs it.

Procedure (E2). `torch.profiler` recorded five training steps after three warm-up steps, for two variants of the same stack. Each operator's own CPU time was assigned to a category. Time spent inside the optimizer's range counts as optimizer. Step wall time not inside any operator counts as framework overhead. We ran E2 twice, back to back, after E1.

Category	Default layer (dropout 0.1), run 1 / run 2	Dropout-free layer, run 1 / run 2
Step time, unprofiled	2.68 s / 2.13 s (764 / 961 tok/s)	1.52 s / 1.30 s (1,352 / 1,581 tok/s)
GEMM (`mm`, `addmm`, `bmm`)	45.4% / 39.1%	65.4% / 65.2%
Dropout mask generation (`bernoulli_`)	**34.5% / 40.7%**	0%
Attention kernel and softmax	1.6% / 1.2%	11.6% / 11.1%
Framework overhead outside operators	8.1% / 9.1%	9.6% / 8.6%
Other elementwise and reductions	5.9% / 5.5%	3.7% / 4.5%
Copies and layout	2.5% / 2.4%	4.4% / 4.4%
Optimizer (AdamW, all its operators)	1.3% / 1.4%	2.4% / 2.3%
Activation (ReLU or GELU)	0.3% / 0.3%	2.1% / 2.5%
LayerNorm	0.4% / 0.4%	0.8% / 1.4%

Table 7. E2: share of profiled step time, 12 threads, on battery. The default variant's attention runs through batched GEMMs (counted under GEMM) because attention dropout rules out the fused kernel.

What E1 and E2 settle.

1. The paper's original question, "is the sustained GEMM rate near 434 or near 118?", has a clear answer. On battery it is **265 to 373 GFLOP/s**, and it drifts down as the chip heats.
2. The largest software lever on this stack was not fusion but a default: dropout. Its mask generation runs on one thread.
3. With dropout off, two thirds of the step is GEMMs at about 60% of the square-GEMM rate. Fusion (E3) can recover at most the remaining non-GEMM third.
4. Threading is best left to PyTorch's default on Windows. Pinning to any subset of cores lowered clocks and throughput.

What E1 does not yet settle is **how much of the drift and of the pinned sets' low clocks is the battery's power limit**. The mains half of E1 answers that.

---

8. Limitations

This draft synthesises existing literature and two of our own experiments, E1 and E2. Their limits:

- **One evening, battery only.** E1 and E2 ran on battery in one evening. The mains half of E1 is pending, so we cannot yet separate the battery's power limit from Windows scheduling.
- **Drift.** The machine drifted by up to 27% within one sweep, so differences under about 25% between E1's core sets are within the drift, and set order is a confound. The default set's repeat at the end, still faster

than every pinned set, is the main guard against reading order as a pinning effect.

- **Approximate power.** Power is whole-system battery discharge, smoothed by the battery firmware, not CPU package power. Windows exposes no package-power counter without a third-party driver.
- **Profiler distortion.** Profiling changed step time by −3% to +16% and may inflate small operators.
- **No embedding or LM head.** The 19M stack has neither; a real model would add a vocabulary GEMM and a softmax.
- **Unverified hardware details.** We have not verified the laptop's memory speed, its OEM power limits, or its exact instruction set (the vendor page omits AVX-VNNI).

Much of the cited evidence comes from GPUs, including the operator breakdown that anchors Section 3.1. Much of the CPU evidence comes from vendors: Intel, ThirdAI, AWS, Google, DeepSpeed. Several claims rest on sources our notes mark as not re-fetched or secondary (†). These include the Chinchilla rule, Muon's FLOP savings, the optimizer-benchmark critique, NVFP4's results and the on-policy distillation ratios. The theoretical peak estimate rests on unverified FMA widths, and the bandwidth figures are our arithmetic from memory speeds.

We found no CPU training measurements for MoE, PKM, memory layers, FFF, MeZO, GaLore or 8-bit Adam. Every CPU-transfer claim for those methods is inference. Our evaluation context is narrow: one laptop class and one domain (maths word problems). Conclusions may differ on AVX-512 or AMX hardware, where bf16 changes the arithmetic picture, or on Apple M4, where SME changes the fp32 ceiling.

---

9. Conclusion

The question "what makes CPU training cheap" turns out to have two separate answers that the literature usually blurs. Methods marketed as CPU-efficient almost all make *inference* cheap, because inference can exploit fixed weights, hard routing and integer arithmetic. Training cannot: it needs a dense weight gradient, soft or differentiable routing, and full-precision optimizer state. Grove is a case in point. Its ternary storage and one-leaf tree make inference cheap, and neither changes the cost of training. What makes CPU *training* cheaper is unglamorous by comparison: moving fewer bytes per step, wasting fewer cycles to power limits and hybrid-core stragglers, and needing fewer tokens to reach the target. That reframing matters for design. A CPU-first model should be judged by bytes moved per useful token and by tokens needed to reach target quality, not by multiply-accumulates avoided.

Our own profile adds a third, more mundane answer: check the framework's defaults. On this laptop, PyTorch's default transformer layer spends a third or more of each training step drawing dropout masks on one thread. Turning dropout off nearly doubled throughput, more than any method in our taxonomy promises for training.

The second implication is strategic. At about 1/600 to 1/1,500 of an A100, and with marginal electricity costs plausibly matching GPU rental for the same FLOPs, a laptop is not a place to pretrain anything larger than about 30M parameters. It is, however, an excellent place to run everything around training: generating and filtering narrow synthetic data, caching distillation targets, verifying and reranking outputs, and fine-tuning small models against checkers. cpulm's jump from 13.4% to 18.0% SVAMP came from a reranker, not more pretraining, which fits that pattern. E1 and E2 shrank the gap between our training rate and the chip's best GEMM rate from about five-fold to about two-fold. The open empirical question is how much of the rest software can recover and how much the 15 W envelope takes. The mains half of E1 and experiments E3, E6, E9 and E11 can settle it within a week of laptop time.

References

Where a reference link is a mirror, reprint or secondary source, the entry says so. † marks items not re-verified against the primary source during this research.

[1] Williams, Waterman, Patterson. Roofline: an insightful visual performance model. CACM 2009. https://dl.acm.org/doi/10.1145/1498765.1498785 [2] Ivanov et al. Data Movement Is All You Need. MLSys 2021. https://ar5iv.arxiv.org/html/2007.00072 [3] PyTorch Performance Tuning Guide. https://docs.pytorch.org/tutorials/recipes/recipes/tuning_guide.html [4] Intel Core Ultra 7 255U specifications. https://www.intel.com/content/www/us/en/products/sku/241860/intel-core-ultra-7-processor-255u-12m-cache-up-to-5-20-ghz/specifications.html [5] Notebookcheck, Intel Core Ultra 7 255U. https://www.notebookcheck.net/Intel-Core-Ultra-7-255U-Processor-Benchmarks-and-Specs.943065.0.html [6] Wikipedia, Arrow Lake (microprocessor). https://en.wikipedia.org/wiki/Arrow_Lake_(microprocessor) [7] Chips and Cheese, Update on Meteor Lake DRAM latency measurements. https://chipsandcheese.com/p/update-on-meteor-lake-dram-latency-measurements [8] Chips and Cheese, Comparing Crestmonts: No L3 Hurts. https://chipsandcheese.com/p/comparing-crestmonts-no-l3-hurts [9] Chips and Cheese, Analyzing Lion Cove's memory subsystem. https://chipsandcheese.com/p/analyzing-lion-coves-memory-subsystem [10] Intel oneMKL Developer Guide, Managing Performance with Heterogeneous Cores. https://www.intel.com/content/www/us/en/docs/onemkl/developer-guide-linux/2025-2/managing-performance-with-heterogeneous-cores.html [11] Coderx7, PyTorch CPU threading notes (gist)†. https://Gist.github.com/Coderx7/bb7cad895378d9faeedb6118abda1685 [12] TechPowerUp, Intel releases Arrow Lake and Lunar Lake ISA reference guide (2023)†. https://techpowerup.com/310776/intel-releases-arrow-lake-and-lunar-lake-instruction-set-reference-guide [13] Wikipedia, Advanced Matrix Extensions. https://en.wikipedia.org/wiki/Advanced_Matrix_Extensions [14] Intel, Accelerate PyTorch Training and Inference using Intel AMX (vendor). https://www.intel.com/content/www/us/en/developer/articles/technical/accelerate-pytorch-training-inference-on-amx.html [15] TechPowerUp, AMD Strix Point Zen 5 256-bit FPU datapaths. https://techpowerup.com/324873/amd-strix-point-soc-zen-5-and-zen-5c-cpu-cores-have-256-bit-fpu-datapaths [16] Remke, Breuer. Hello SME! arXiv 2409.18779. https://arxiv.org/pdf/2409.18779 [17] Hugging Face, Efficient Training on CPU. https://huggingface.co/docs/transformers/en/perf_train_cpu [18] PyTorch blog, Empowering PyTorch on Intel Xeon Scalable processors with Bfloat16 (vendor-linked). https://pytorch.org/blog/empowering-pytorch-on-intel-xeon-scalable-processors-with-bfloat16 [19] Hugging Face, Accelerating PyTorch Transformers with Intel Sapphire Rapids, part 1 (vendor co-authored). https://huggingface.co/blog/intel-sapphire-rapids [20] Intel/TUM, LLM research report (vendor-linked)†. https://cdrdv2-public.intel.com/866175/p1079012-intel-adhoc-b2b-large-language-models-research-report-eng-v8.pdf [21] Micikevicius et al. Mixed Precision Training. arXiv 1710.03740†. https://arxiv.org/abs/1710.03740 [22] Peng et al. FP8-LM. arXiv 2310.18313. https://arxiv.org/abs/2310.18313 [23] Fishman et al. Scaling FP8 training to trillion-token LLMs. arXiv 2409.12517†. https://arxiv.org/abs/2409.12517 [24] Wortsman et al. Stable and low-precision training for large-scale vision-language models (SwitchBack). arXiv 2304.13013. https://arxiv.org/abs/2304.13013 [25] Xi et al. Jetfire: INT8 transformer pretraining. ICML 2024†. https://proceedings.mlr.press/v235/xi24b.html [26] Xi et al. Training transformers with 4-bit integers. arXiv 2306.11987†. https://arxiv.org/abs/2306.11987 [27] NVIDIA. Pretraining LLMs with NVFP4. arXiv 2509.25149†. https://arxiv.org/abs/2509.25149v1 [28] Ma et al. BitNet b1.58 2B4T Technical Report. arXiv 2504.12285. https://arxiv.org/html/2504.12285 [29] Kaushal et al. Spectra: ternary, quantized and FP16 LMs (TriLM). arXiv 2407.12327. https://arxiv.org/abs/2407.12327 [30] Zhu et al. Scalable MatMul-free Language Modeling. arXiv 2406.02528. https://arxiv.org/abs/2406.02528 [31] Wang et al. 1-bit AI Infra Part 1.1 (bitnet.cpp). arXiv 2410.16144. https://arxiv.org/html/2410.16144v2 [32] AlphaNeural, Chimera 5.1 model page (self-reported)†. https://app.alphaneural.io/models/Lgr54HFi/chimera [33] Fedus et al. Switch Transformers. arXiv 2101.03961. https://arxiv.org/abs/2101.03961v1 [34] Dai et al. DeepSeekMoE. arXiv 2401.06066. https://www.arxiv.org/pdf/2401.06066 [35] SonicMoE. arXiv 2512.14080. https://arxiv.org/html/2512.14080v1 [36] Tan et al. ScatterMoE. arXiv 2403.08245. https://arxiv.org/pdf/2403.08245v2 [37] Roller et al. Hash Layers for Large Sparse Models. NeurIPS 2021. https://arxiv.org/abs/2106.04426v2 [38] Belcak, Wattenhofer. Fast Feedforward Networks. arXiv 2308.14711. https://ar5iv.labs.arxiv.org/html/2308.14711 [39] Belcak, Wattenhofer. Exponentially Faster Language Modelling (UltraFastBERT). arXiv 2311.10770. https://arxiv.org/html/2311.10770v2 [40] Lample et al. Large

Memory Layers with Product Keys. NeurIPS 2019. https://ar5iv.arxiv.org/html/1907.05242 [41] Berges et al. Memory Layers at Scale. arXiv 2412.09764. https://arxiv.org/html/2412.09764v2 [42] Chen et al. SLIDE. MLSys 2020. https://ar5iv.labs.arxiv.org/html/1903.03129 [43] Daghaghi et al. Accelerating SLIDE Deep Learning on Modern CPUs. MLSys 2021. https://ar5iv.labs.arxiv.org/html/2103.10891 [44] Chen et al. MONGOOSE. ICLR 2021. https://iclr.cc/virtual/2021/poster/3277 [45] AWS ML Blog, Accelerating large-scale neural network training on CPUs with ThirdAI and AWS Graviton (vendor/partner). https://aws.amazon.com/blogs/machine-learning/accelerating-large-scale-neural-network-training-on-cpus-with-thirdai-and-aws-graviton [46] ThirdAI BOLT2.5B press release, reprint (vendor). https://journalofcyberpolicy.com/?p=153034 [47] Zeng et al. LookupFFN. ICML 2023. https://proceedings.mlr.press/v202/zeng23a.html [48] Blalock, Guttag. Multiplying Matrices Without Multiplying (MADDNESS). ICML 2021. https://proceedings.mlr.press/v139/blalock21a/blalock21a.pdf [49] Sun et al. meProp. ICML 2017. http://proceedings.mlr.press/v70/sun17c/sun17c.pdf [50] SWAT: Sparse Weight Activation Training. arXiv 2001.01969. https://arxiv.org/pdf/2001.01969 [51] Rae et al. Scaling Language Models: Gopher. arXiv 2112.11446. https://arxiv.org/pdf/2112.11446 [52] Malladi et al. Fine-Tuning Language Models with Just Forward Passes (MeZO). NeurIPS 2023. https://arxiv.org/abs/2305.17333 [53] Zhang et al. Revisiting Zeroth-Order Optimization for Memory-Efficient LLM Fine-Tuning (ZO-Bench). ICML 2024. https://arxiv.org/html/2402.11592v3 [54] Synthetic Benchmarks Overstate Forward-Forward Scaling (secondary host)†. https://newx.sg/paper/detail/adc0104c-62fa-11f1-b84c-00163e10baa7 [55] Hu et al. LoRA. arXiv 2106.09685†. https://arxiv.org/abs/2106.09685 [56] Dettmers et al. QLoRA. arXiv 2305.14314†. https://arxiv.org/abs/2305.14314 [57] Lenovo Press LP2179, Fine-tuning LLMs using Intel Xeon CPUs (vendor). https://lenovopress.lenovo.com/lp2179-fine-tuning-llms-using-intel-xeon-cpus [58] ss8319, PEFT of Llama-3-8B on Xeon with AMX (blog). https://ss8319.github.io/blog-post-2 [59] Arabpour et al. LoRA Fine-Tuning Without GPUs. arXiv 2507.01806. https://arxiv.org/abs/2507.01806 [60] LoRA/QLoRA on consumer GPUs. arXiv 2509.12229. https://arxiv.org/abs/2509.12229 [61] Eldan, Li. TinyStories. arXiv 2305.07759. https://arxiv.org/abs/2305.07759 [62] Liu et al. TinyGSM. arXiv 2312.09241. https://arxiv.org/abs/2312.09241 [63] Gunasekar et al. Textbooks Are All You Need (Phi-1). arXiv 2306.11644†. https://arxiv.org/abs/2306.11644 [64] Geiping, Goldstein. Cramming. arXiv 2212.14034. https://arxiv.org/html/2212.14034 [65] Jordan et al. modded-nanogpt speedrun README. https://github.com/KellerJordan/modded-nanogpt [66] Liu et al. Muon is Scalable for LLM Training. arXiv 2502.16982†. https://arxiv.org/abs/2502.16982 [67] Wen, Hall, Ma, Liang. Fantastic pretraining optimizers and where to find them. ICLR 2026†. https://arxiv.org/html/2509.02046v2 [68] Hoffmann et al. Training Compute-Optimal LLMs (Chinchilla). arXiv 2203.15556†. https://arxiv.org/abs/2203.15556 [69] Pearce, Song. Reconciling Kaplan and Chinchilla Scaling Laws. arXiv 2406.12907†. https://arxiv.org/abs/2406.12907 [70] Porian et al. Resolving Discrepancies in Compute-Optimal Scaling. arXiv 2406.19146†. https://arxiv.org/abs/2406.19146 [71] Muennighoff et al. Scaling Data-Constrained Language Models. arXiv 2305.16264. https://arxiv.org/abs/2305.16264 [72] Findings of the BabyLM Challenge (2023). arXiv 2504.08165. https://arxiv.org/abs/2504.08165 [73] Hu et al. Findings of the Second BabyLM Challenge. arXiv 2412.05149. https://arxiv.org/abs/2412.05149 [74] Charpentier, Samuel. GPT or BERT: why not both? arXiv 2410.24159. https://arxiv.org/abs/2410.24159 [75] Findings of the Third BabyLM Challenge. ACL Anthology 2025. https://aclanthology.org/2025.babylm-main.28 [76] Timiryasov, Tastet. Baby Llama. arXiv 2308.02019†. https://arxiv.org/abs/2308.02019 [77] Maginative, Thinking Machines on-policy distillation summary (secondary)†. https://www.maginative.com/thinking-machines-claims-30x-cost-cut-for-training-ai-models [78] Dettmers et al. 8-bit Optimizers via Block-wise Quantization. arXiv 2110.02861. https://arxiv.org/abs/2110.02861 [79] Shazeer, Stern. Adafactor. arXiv 1804.04235†. https://arxiv.org/abs/1804.04235 [80] Chen et al. Symbolic Discovery of Optimization Algorithms (Lion). arXiv 2302.06675†. https://arxiv.org/abs/2302.06675 [81] Lv et al. Full Parameter Fine-tuning with Limited Resources (LOMO). arXiv 2306.09782. https://arxiv.org/abs/2306.09782 [82] Zhao et al. GaLore. arXiv 2403.03507. https://arxiv.org/abs/2403.03507 [83] SparseLoRA (GaLore comparison appendix). arXiv 2506.16500. https://arxiv.org/pdf/2506.16500 [84] Chen et al. Training Deep Nets with Sublinear Memory Cost. arXiv 1604.06174†. https://arxiv.org/abs/1604.06174 [85] Ren et al. ZeRO-Offload. arXiv 2101.06840.

https://arxiv.org/abs/2101.06840 [86] PyTorch dev-discuss, TorchInductor Update 6: CPU backend. https://dev-discuss.pytorch.org/t/torchinductor-update-6-cpu-backend-performance-update-and-new-features-in-pytorch-2-1/1514 [87] intel-extension-for-pytorch issue #867 (EOL announcement). https://github.com/intel/intel-extension-for-pytorch/issues/867 [88] DeepSpeed ZeRO-Offload tutorial (vendor). https://www.deepspeed.ai/tutorials/zero-offload/ [89] llama.cpp PR #8669, mirror. https://app.semanticdiff.com/gh/ggml-org/llama.cpp/pull/8669/overview [90] llama.cpp training-support commit 10d2af0, mirror. https://cdn04132025.gitlink.org.cn/replica/llama.cpp/commit/10d2af0eaa0aafd7c6577b279dfa5221ff44a63f [91] llama.cpp examples/training README, Debian copy. https://sources.debian.org/src/llama.cpp/8064%2Bdfsg-1/examples/training/README.md [92] llama.cpp issue #18499, mirror. https://www.goodfirstissue.org/ggml-org/llama.cpp/issues/18499 [93] Karpathy, llm.c README. https://github.com/karpathy/llm.c/blob/master/README.md [94] Karpathy, llm.c Discussion #481 (GPT-2 124M reproduction). https://github.com/karpathy/llm.c/discussions/481 [95] Karpathy, nanoGPT README. https://github.com/karpathy/nanoGPT/blob/master/README.md [96] InsiderLLM, MLX LoRA guide (low-quality source)†. https://insiderllm.com/guides/fine-tuning-mac-lora-mlx/ [97] TechPowerUp, Intel MLPerf v2.1 results (vendor press release reprint). https://techpowerup.com/300952/intel-delivers-leading-ai-performance-results-on-mlperf-v2-1-industry-benchmark-for-dl-training [98] DigiTimes, MLPerf v3.0 results (vendor press release reprint). https://apps.digitimes.com/news/a20230629PR202.html [99] Intel newsroom, MLPerf Training v3.0 (vendor). https://www.intel.com/content/www/us/en/newsroom/news/intel-gaudi-ai-accelerator-brings-greater-ai-choice.html [100] The Register, The troublesome economics of CPU-only AI (2024). https://www.theregister.com/2024/10/29/cpu_gen_ai_gpu/ [101] Hugging Face, Fine-tuning Stable Diffusion Models on Intel CPUs (vendor co-authored). https://huggingface.co/blog/stable-diffusion-finetuning-intel [102] Intel Analytics Software, Why Pay More for Machine Learning? (vendor). https://medium.com/intel-analytics-software/why-pay-more-for-machine-learning-893683bd78e4 [103] Hugging Face forum, nanoGPT training costs across A100, H100 and RTX6000. https://discuss.huggingface.co/t/benchmarked-nanogpt-training-costs-across-a100-h100-and-rtx6000-a100-was-2x-more-efficient-than-h100/171443 [104] Le et al. Building High-level Features Using Large Scale Unsupervised Learning. ICML 2012. https://arxiv.org/html/1112.6209